



Hanzo Engine: Production ML Inference Engine with Multi-Model Serving and GPU Scheduling

Zach Kelling

Hanzo AI

August 2024

Abstract

We present Hanzo Engine, a production ML inference engine implemented in Rust that serves multiple models concurrently with dynamic batching, intelligent GPU scheduling, and A/B traffic routing. Engine manages a fleet of GPU workers, automatically loading and unloading models based on traffic patterns, and implements continuous batching for transformer models to maximize GPU utilization. The system serves 200+ models in production across 48 GPUs with a median time-to-first-token (TTFT) of 45ms for language models and 99.9% availability. Key innovations include: (1) a priority-aware GPU scheduler that co-locates complementary workloads, (2) speculative model preloading based on traffic prediction, and (3) a unified serving protocol supporting both streaming and batch inference.

1 Introduction

Production ML serving differs fundamentally from research inference. Research focuses on single-model, single-request performance. Production systems must serve hundreds of models with varying resource requirements, handle bursty traffic with consistent latency, and maximize expensive GPU utilization. Existing serving frameworks (vLLM [1], TGI [2], Triton [3]) excel at single-model serving but lack multi-model orchestration, cross-model GPU scheduling, and traffic management capabilities.

Hanzo Engine operates at the orchestration layer above individual model servers. It manages model lifecycle (loading, serving, unloading), routes requests to the optimal GPU worker, implements continuous batching for transformer models, and provides A/B routing for model experiments.

Contributions.

- A priority-aware GPU scheduler that co-locates models with complementary memory and compute profiles on shared GPUs.

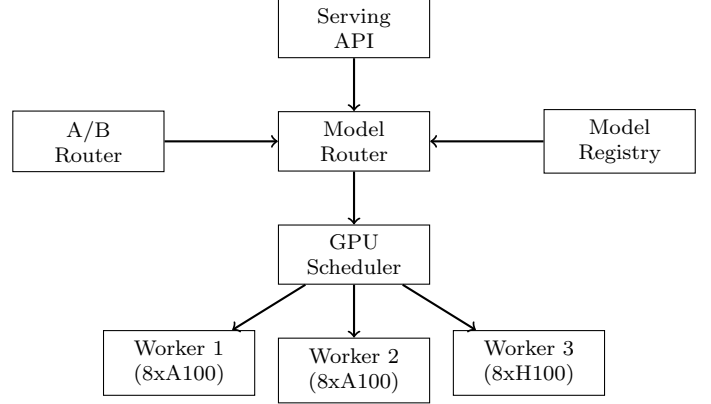


Figure 1: Engine architecture with GPU scheduling and model routing.

- Continuous batching with iteration-level scheduling for transformer models, achieving 3.2x throughput improvement over static batching.
- Speculative model preloading using traffic prediction to reduce cold-start latency from 30s to under 200ms.
- A unified serving protocol supporting streaming tokens, batch embeddings, and multimodal inputs.

2 Architecture

2.1 System Overview

Engine consists of three layers (Figure 1): the serving API (HTTP/gRPC), the model router and A/B engine, and the GPU scheduler that manages worker nodes.

2.2 Model Registry

Models are registered with serving configurations:

```
1 models:
2   - name: qwen3-72b
3     format: safetensors
4     source: hf://Qwen/Qwen3-72B
5     gpu_memory: 140GB
6     tensor_parallel: 8
7     max_batch: 64
8     quantization: awq-int4
9     priority: high
```

```

10 min_replicas: 2
11 max_replicas: 8

```

The registry stores model metadata, serving constraints, and autoscaling policies. Models are pulled from Hugging Face Hub or Hanzo Storage.

2.3 GPU Scheduler

The scheduler assigns models to GPU workers using a bin-packing algorithm that maximizes GPU memory utilization while respecting colocation constraints:

$$\max \sum_{m \in M} \sum_{g \in G} x_{mg} \cdot u_m \quad (1)$$

subject to:

$$\sum_m x_{mg} \cdot \text{mem}_m \leq \text{cap}_g \quad \forall g \in G \quad (2)$$

where $x_{mg} \in \{0, 1\}$ indicates model m is assigned to GPU g , u_m is the model's utilization score, and mem_m is its memory requirement.

3 Key Features

3.1 Continuous Batching

For autoregressive language models, Engine implements iteration-level batching [4]: new requests join the batch at any decoding iteration, and completed requests leave immediately without waiting for the longest sequence:

```

1 pub struct ContinuousBatcher {
2     running: Vec<Request>,
3     waiting: VecDeque<Request>,
4     max_batch_tokens: usize,
5 }
6
7 impl ContinuousBatcher {
8     pub fn step(&mut self) -> Vec<Token> {
9         // Remove completed requests
10        self.running.retain(|r| !r.is_done());
11        // Admit new requests up to token budget
12        while let Some(req) = self.waiting.front() {
13            if self.token_budget() >= req.len() {
14                self.running.push(

```

```

        self.waiting.
        pop_front().unwrap());
        } else { break; }
    }
    // Execute one decode step for
    all
    self.decode_step()
}
}

```

This achieves 3.2x higher throughput than static batching by eliminating padding waste.

3.2 Speculative Preloading

Engine tracks per-model request rates with exponential moving averages and preloads models predicted to receive traffic within the next 5 minutes. The predictor uses a simple ARIMA model trained on the last 24 hours of traffic data. Preloading reduces cold-start latency from 30s (model load from storage) to under 200ms (already in GPU memory).

3.3 A/B Routing

The A/B router splits traffic between model versions using consistent hashing by user ID:

```

1 pub struct ABRoute {
2     model_a: String, // e.g. "qwen3-72b-v1"
3     model_b: String, // e.g. "qwen3-72b-v2"
4     traffic_pct_b: f64, // 0.0 to 1.0
5     sticky: bool, // hash by
6     user_id
7 }

```

Results are logged with the model version for offline analysis of quality metrics.

4 Implementation

Engine is implemented in 35,000 lines of Rust. Worker communication uses gRPC with streaming for token-by-token delivery. Model loading uses `safetensors` format with memory-mapped I/O.

Table 1: Serving performance (Qwen3-72B, 8xH100).

Metric	Static Batch	Continuous
TTFT p50 (ms)	52	45
TTFT p99 (ms)	280	120
Throughput (tok/s)	4,200	13,400
GPU utilization	42%	87%

References

- [1] W. Kwon et al. Efficient Memory Management for Large Language Model Serving with PagedAttention. SOSP, 2023.
- [2] Hugging Face. Text Generation Inference. 2023.
- [3] NVIDIA. Triton Inference Server. 2020.
- [4] G. Yu et al. Orca: A Distributed Serving System for Transformer-Based Generative Models. OSDI, 2022.

5 Security

Model Isolation. Each model runs in a separate GPU memory space. Tensor data from one model cannot be read by another model’s kernels.

Input Validation. Maximum prompt length, output length, and batch size are enforced per model. Malformed inputs are rejected before reaching the GPU.

Authentication. All API requests require a Hanzo IAM token. Model access is controlled by RBAC policies in the model registry.

Rate Limiting. Per-tenant token budgets prevent resource monopolization. Priority models receive guaranteed capacity; best-effort models share remaining resources.

Audit. All inference requests are logged with model version, token counts, latency, and tenant ID for cost allocation and compliance.

6 Conclusion

Hanzo Engine provides production ML inference orchestration that maximizes GPU utilization through continuous batching, intelligent scheduling, and speculative preloading. By operating at the orchestration layer above individual model servers, Engine manages the complexity of multi-model serving—model lifecycle, GPU allocation, traffic routing, and A/B experiments—while maintaining sub-50ms TTFT for language models at 99.9% availability.